Conceptos basicos

CPU, memoria y registros

- La **memoria** es como una calle larguísima de “casillas” numeradas.
- El CPU no trabaja directo con “ideas”, trabaja con **números** y **direcciones**.
- Los **registros** son cajitas súper rápidas dentro del CPU.

Registros principales

- **AX**: acumulador (muy usado para operaciones)
- **BX**: base (útil para direcciones)
- **CX**: contador (loops)
- **DX**: datos (y para operaciones como imprimir en DOS)
- **SP/BP**: stack (pila)
- **SI/DI**: índices (manejo de arreglos/memoria)

AX = AH (alto 8 bits) + AL (bajo 8 bits)

Segmentos

En TASM (DOS), una dirección real se arma como:

Dirección física = segmento * 16 + desplazamiento

Segmentos típicos:

- **CS** = segmento de código
- **DS** = segmento de datos
- **SS** = segmento de pila

Ejemplo de un programa

```
.MODEL SMALL
.STACK 100h
```

.DATA
msg db "Hola TASM!$"

.CODE
main PROC
mov ax, @data
mov ds, ax

```
mov ah, 09h
lea dx, msg
int 21h

mov ah, 4Ch
int 21h
```

main ENDP
END main

- `.MODEL SMALL`: 1 segmento para código y 1 para datos (lo más común para aprender).
- `.STACK 100h`: reserva 256 bytes para pila.
- `.DATA`: aquí van variables.
- `.CODE`: aquí va el programa.
- `mov ax, @data` + `mov ds, ax`: **configura DS** para que tus variables funcionen.
- `int 21h` con `ah=09h`: imprime string terminado en `$`.
- `int 21h` con `ah=4Ch`: salir del programa.

Instrucciones comunes

`MOV` (copiar)

`mov destino, fuente`

Ejemplos:

- `mov ax, 5` → AX = 5
- `mov bx, ax` → BX = AX

`LEA` (Load Effective Address)

`lea dx, msg`

No carga el texto, carga **la dirección** donde está `msg` en memoria (para que DOS pueda leerlo).

`INT 21h` (servicios de DOS)

Es como llamar una "función" del sistema operativo DOS.

Direcciones

El procesador 8086 solo tiene registros de 16 bits.

Un registro de 16 bits solo puede guardar:

0000h → FFFFh

Dirección Física = (Segmento × 16) + Offset

Segmento = 1000h
Offset = 0020h

1000h × 10h = 10000h
10000h + 0020h = 10020h
Dirección física final: **10020h**

Que es el segmento?

El segmento es un registro especial que apunta a una zona de memoria.

Registro	Uso
CS	Código
DS	Datos
SS	Stack
ES	Extra

Uso en la practica

mov ax, @data
mov ds, ax

Estás diciendo:

> DS ahora apunta al segmento donde están mis variables.

Entonces cuando haces:

mov al, variable

El procesador realmente está usando:

DS:offset_de_variable

Internamente calcula:

(DS × 16) + offset

Tipos de direccionamiento en x86

Direccionamiento inmediato

El dato está en la instrucción.

mov ax, 5

AX = 5

No hay memoria involucrada.

Direccionamiento por registro

Se copia entre registros.

mov ax, bx

No toca memoria.

Direccionamiento directo (memoria directa)

mov al, variable

Significa:

AL = contenido_en_DS:offset(variable)

Direccionamiento indirecto (muy importante)

Aquí usamos registros como punteros.

mov si, offset variable
mov al, [si]

Esto significa:

AL = contenido_en_DS:SI

Las llaves significan: "Ir a memoria"

Direccionamiento basado

mov al, [bx]

Usa DS:BX como dirección.

Direccionamiento indexado

mov al, [si]
mov al, [di]

Usado para recorrer arreglos.

Direccionamiento basado + indexado

mov al, [bx + si]

Dirección efectiva:

DS:(BX + SI)

Con desplazamiento

mov al, [bx + 5]

Direccion:

DS:(BX + 5)

¿Qué es el OFFSET realmente?

Si tienes:

```
variable db 10
```

Y está en el segmento DS, en posición:

```
DS:0030h
```

Entonces:

```
offset variable = 0030h
```

El offset es el desplazamiento **dentro del segmento**.

Ejemplo completo

Imagina:

```
DS = 2000h
variable está en offset 0010h
```

Entonces:

```
Dirección física = 2000h × 10h + 0010h
                 = 20000h + 0010h
                 = 20010h
```

Ahí está guardado tu dato.

Stack

El stack usa:

```
SS:SP
```

Cuando haces:

```
push ax
```

Internamente usa:

```
SS:(SP - 2)
```

El stack crece hacia abajo.

Lo más importante que debes recordar

Cuando ves:

```
mov al, [bx]
```

Eso significa:

```
AL = contenido en memoria en DS:BX
```

Cuando ves:

```
mov al, variable
```

Eso significa:

```
AL = contenido en DS:offset(variable)
```